Linköping University | Department of Computer and Information Science
Master's thesis, 30 ECTS | Computer Science
26 | LIU-IDA/LITH-EX-A--26/033--SE

Behaviour Tree Visualisation for Autonomous Aircraft Simulation

Visualisering av beteendeträd för simulering av autonoma flygplan

Niklas Roslund

Supervisor : Daniel de Leng
Examiner : Tom Ziemke

External supervisor : Torvald Mårtensson

LINKÖPING UNIVERSITY

Linköpings universitet
SE–581 83 Linköping
+46 13 28 10 00 , www.liu.se

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida `http://www.ep.liu.se/`.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: `http://www.ep.liu.se/`.

© Niklas Roslund

Abstract

Simulation is often used when developing autonomous aircraft, since it makes it possible to run the same scenario several times and inspect parts of the system that would be difficult to see during real flight. However, it can be hard for engineers to understand why an autonomous aircraft behaved in a certain way, especially when relevant information is spread between logs, spatial views, internal states, and decision logic. This thesis investigates how visualisation can support analysis of internal autonomy processes in autonomous aircraft simulation. The work was done in three steps: two pre-study group interviews with engineers, development of a visualisation prototype, and a qualitative evaluation using simulation scenarios. The prototype was built with ROS2, Gazebo, PX4, RViz, and Behaviour Trees. It combines a spatial view of the aircraft and mission with visualisation of Behaviour Tree execution and selected internal values. The results suggest that the most useful support came from combining several views rather than from one single visual element. In the evaluation, the participants used the spatial view, Behaviour Tree viewer, and Behaviour Tree heads-up display together when reasoning about both normal and anomalous behaviour. Without the internal visualisation, they could still make reasonable guesses about possible errors, but had less support for checking those guesses. The study is exploratory and limited, but indicates that visualising internal autonomy states can make simulation-based analysis more grounded and less dependent on unsupported assumptions.

Acknowledgments

First, I want to thank my supervisor, Daniel de Leng for guidance, feedback, and support throughout the project. I would also like to thank my examiner, Tom Ziemke.

A special thanks goes to Torvald Mårtensson and Saab for making the thesis possible and for providing the industrial context for the work. I also want to thank the participants who took part in the interviews.

Niklas Roslund
Linköping, June, 2026

Contents

Abstract iii

Acknowledgments iv

Contents v

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Aim . . . 2
- 1.3 Research Questions . . . 2
- 1.4 Approach . . . 3
- 1.5 Contributions . . . 3
- 1.6 Delimitations . . . 3

2 Background 5
- 2.1 Autonomous Aircraft and Simulation Based Development . . . 5
- 2.2 Autonomy Architectures and Decision Making . . . 7
- 2.3 Behaviour Trees in Autonomous Systems . . . 7
- 2.4 Simulation Ecosystems for Autonomous UAV Development . . . 9

3 Related Work 11
- 3.1 Transparency and Explainability in Autonomous Systems . . . 11
- 3.2 Visualisation for Robotic Debugging and System Analysis . . . 12
- 3.3 Tracing and Monitoring in ROS Systems . . . 13
- 3.4 Behaviour Trees and Execution Analysis . . . 14
- 3.5 Visualisation of Uncertainty . . . 15

4 Method 16
- 4.1 Research Design . . . 16
- 4.2 Pre-study Interviews . . . 17
- 4.3 Prototype Development . . . 18
- 4.4 Evaluation Method . . . 19
- 4.5 Ethical Considerations . . . 21

5 Result 22
- 5.1 Findings from the Pre-Study Interviews . . . 22
- 5.2 Prototype Implementation . . . 24
- 5.3 Evaluation Results . . . 26

6 Discussion 31

6.1 Linked Views and Behaviour Tree . . . 31
6.2 Interpretation and Validation of Explanations . . . 32
6.3 Temporal Analysis and Replay . . . 32
6.4 Information Density and Task Dependent Views . . . 33
6.5 Scope and Methodological Limitations . . . 33
6.6 The Work in a Wider Context . . . 35
6.7 Future Work . . . 35

7 Conclusion **37**
7.1 Research Question 1 . . . 37
7.2 Research Question 2 . . . 38
7.3 Research Question 3 . . . 38
7.4 Overall . . . 39

Bibliography **40**

A Interview Questions **44**
A.1 Pre-Study Interview Questions . . . 44
A.2 Evaluation Session Questions . . . 44

List of Figures

2.1 Simplified Behaviour Tree for an autonomous aircraft mission. The Sequence node represents the main mission order, where the aircraft first takes off, then selects between avoiding a keep-out zone or following the planned path, and finally lands. 8

5.1 Spatial view from the prototype in RViz. The figure shows the aircraft path, flown path, and no fly zones used in the first keep-out scenario. 25
5.2 Full prototype view used in the third evaluation scenario. The upper part shows Behaviour Tree execution, the centre shows the spatial scene in RViz, and the right side shows the BT HUD with selected internal state related information. 26
5.3 Simplified sketch of the nominal scenario. The planned path (red) and the actual path (black) are shown together with the no fly zones. 27
5.4 Anomalous scenario shown with limited visual support. The aircraft behaviour can be observed in the scene, but the additional internal decision related views are not available in the same way as in the full prototype view. 28

List of Tables

5.1 Main themes identified in the pre-study interviews. 23

1 Introduction

Autonomous aircraft systems are increasingly developed and validated through simulation [1]. Simulation environments allow engineers to explore system behaviour under controlled and repeatable conditions, while providing access to internal system state that is otherwise difficult or impossible to observe in real world operation. As autonomy increases, understanding and analysing system behaviour becomes more challenging because decisions depend on internal representations, uncertain estimates, and distributed decision processes.

This thesis addresses these challenges by investigating how visualisation can support engineers in analysing autonomous aircraft behaviour during simulation based development. The focus is not on piloted operation or real time control, but on understanding how internal autonomy processes, especially Behaviour Tree (BT) execution, can be made visible and interpretable during testing and evaluation.

1.1 Motivation

Simulation plays a central role in the development, integration, and verification of autonomous aircraft systems [1]. Modern simulation environments generate large volumes of data describing both external motion and internal system processes, including state estimation, perception, planning, and control. While this data is essential for understanding system behaviour, it is often distributed across multiple subsystems and data streams, making analysis complex and cognitively demanding.

In the context of piloted aircraft, simulation visualisation tools have historically been designed to support human operators by replicating external flight cues and presenting information relevant for piloting tasks [2]. Such visualisations prioritise perceptual realism and situational awareness for pilots. Autonomous aircraft development creates different analysis needs. Engineers are not only concerned with how the aircraft appears to fly, but with how internal decisions are formed, how uncertainty propagates through the system, and why particular actions are selected.

Autonomous behaviour emerges from interactions between multiple layers of an autonomy architecture, including estimation, planning, decision logic, and control [3, 4]. Sensor meas-

urements are subject to noise and uncertainty, state estimators fuse heterogeneous data sources, behaviour layers select actions or mission states, planners generate and evaluate trajectories, and controllers execute selected commands. When behaviour deviates from expectations, engineers must reconstruct how these layers interacted at a specific moment in time. This reconstruction is difficult because internal decision processes are not directly observable and are often temporally decoupled from externally visible motion. As a result, developers must manually inspect large volumes of log data and scattered visualisation outputs in order to reconstruct how internal autonomy processes produced a particular behaviour.

The difficulty of interpreting autonomous decision making has been widely recognised. Sparrow et al. emphasise that evaluation of autonomous systems must consider the information available to the system and the criteria it optimises, rather than focusing solely on outcomes [5]. Salvini et al. similarly highlight the importance of transparency for accountability and human oversight in safety critical autonomous systems [6]. Together, these perspectives underline the need for tools that expose internal system state and decision relevant information in a structured and interpretable manner.

Visualisation provides a natural mechanism for supporting such analysis. By presenting internal autonomy data in spatial and temporal context, visualisation can reduce cognitive burden and support reasoning about system behaviour [7, 8]. However, existing visualisation tools for autonomous aircraft simulation are often inherited from piloted simulation or general robotics frameworks and are not explicitly designed to support systematic analysis of autonomous decision making. In particular, they rarely connect spatial mission context with the execution state of a Behaviour Tree or similar decision structure. This gap motivates the present thesis.

1.2 Aim

This thesis investigates how visualisation can support analysis of internal autonomy processes in autonomous aircraft simulation. Rather than focusing on improving control algorithms or flight performance, the work examines how internal computational state can be structured and visually represented to support interpretation of autonomous system behaviour during development and testing. In particular, the study considers autonomy related information such as state estimation variables, Behaviour Tree execution state, planner outputs, and relationships between planned and executed actions. Based on insights from literature and interviews with engineers, a prototype visualisation tool is developed to explore how exposing and structuring internal autonomy data can support analysis of simulated system behaviour.

1.3 Research Questions

To guide the work toward this aim, the following research questions have been formulated:

1. Which internal representations within autonomous systems, including Behaviour Tree execution state, are most critical for diagnosing system behaviour in simulation scenarios?
2. How can these internal representations be structured and visualised to support interpretation of autonomous decision processes in simulation?
3. How does a structured visualisation approach support experts in interpreting and reasoning about anomalies in autonomous system behaviour during simulation?

1.4 Approach

This thesis adopts a qualitative, analysis oriented research approach. A literature review is conducted to establish the theoretical and technical background related to autonomous systems, simulation environments, visualisation, and evaluation of autonomous behaviour. This review informs the identification of key challenges and gaps in existing visualisation practices. The study is conducted in three phases: identification of analysis needs, development of a visualisation prototype, and qualitative evaluation through scenario based interaction.

To ground the study in practical development contexts, qualitative interviews are conducted with engineers who work with autonomous aircraft simulation. The interviews focus on identifying typical analysis tasks, information needs, and challenges encountered when using existing visualisation tools. Particular attention is given to tasks such as diagnosing unexpected manoeuvres, analysing mode transitions, comparing planned and executed trajectories, and tracing the causes of anomalous behaviour.

Based on insights from the literature review and interviews, a prototype visualisation tool is developed. The prototype is designed to present autonomy related simulation data in a way that supports process based analysis rather than outcome based assessment alone. The visualisation emphasises internal system state, temporal context, and relationships between autonomy layers.

The prototype is evaluated through scenario based interaction and qualitative assessment. The evaluation focuses on usability, perceived usefulness, and the extent to which the visualisation supports engineers in identifying issues and reasoning about autonomous behaviour. This approach aligns with established practices for evaluating analytical tools in engineering and human computer interaction research.

1.5 Contributions

The intended contributions of this thesis are as follows:

- A prototype visualisation system that demonstrates approaches for exposing internal autonomous system state and behaviour during simulation.
- An empirical identification of key internal autonomy states and information needs, with particular focus on Behaviour Tree execution, relevant for diagnosing autonomous system behaviour.
- Design insights from the development of a visualisation prototype for making internal autonomy processes more interpretable during simulation based analysis.
- A qualitative evaluation of how the proposed visualisation approach supports users in interpreting and reasoning about autonomous system behaviour.

1.6 Delimitations

This thesis focuses on simulation based analysis of autonomous aircraft behaviour during development and testing. It does not address operational deployment, real time pilot interfaces, certification, or formal safety assurance. The work assumes access to simulation environments that provide internal system state and ground truth information, which may not be available in real world operation.

The thesis is limited to analysis oriented visualisation and does not propose autonomous control algorithms or learning based decision making methods. Furthermore, while ethical and safety considerations are discussed at a conceptual level, the thesis does not claim to improve operational safety directly or to provide guarantees regarding system correctness. The scope is deliberately restricted to supporting understanding and analysis during development.

The results should therefore be interpreted as applying to an exploratory Behaviour Tree based simulation prototype, rather than to autonomous aircraft systems in general.

2 Background

This chapter introduces the technical background needed to understand the thesis. It first describes autonomous aircraft and simulation based development, since the prototype is evaluated in a simulated UAV environment. It then introduces autonomy architectures and Behaviour Trees, which are central to the implemented visualisation. Finally, it describes the simulation ecosystem used in the work and why access to internal state matters when analysing autonomous behaviour.

2.1 Autonomous Aircraft and Simulation Based Development

Autonomous aircraft build on ideas from control theory, robotics, artificial intelligence, and dynamic systems. Unlike a conventional aircraft controlled by a pilot, an autonomous aircraft must use sensor information, estimate its own state, predict relevant future states, and select control outputs that support the mission objective. These capabilities can be understood as part of a layered autonomy process, where sensing, state estimation, planning, and control are connected but handled by different parts of the system. This separation supports modular development and testing, while the overall behaviour still depends on how the components interact. Thrun et al. describe robot environment interaction in terms of sensors, controls, state, measurements, and internal belief, which provides a useful basis for understanding this type of autonomy stack [3, pp. 19–27].

A common challenge in autonomous systems is uncertainty. Sensor measurements are noisy, incomplete, and dependent on environmental conditions, which means that the system cannot access the world directly as ground truth. Instead, it must act on internal representations constructed from available sensor data, such as estimated position, velocity, detected objects, maps, or mission relevant constraints. Thrun et al. describe this as a central problem in robotics, where sensors, actuation, internal models, and algorithmic approximations all introduce uncertainty [3, pp. 3–5]. For state estimation, this is often handled through filtering methods that estimate state from noisy measurements, with the Kalman filter being a classical example [9]. This creates an analysis problem, since decisions are made on the basis of what the system believes about the world rather than what is actually true. Uncertainty can therefore propagate through estimation, planning, and control, and influence both what action is selected and how it is executed. Understanding these internal states and how they affect

planning and control output is therefore essential when analysing autonomous behaviour.

Simulation plays a central role in the development and evaluation of autonomous aircraft [1]. Simulated environments can model the aircraft, sensors, and surrounding conditions without exposing hardware or people to the risks and costs of repeated flight tests [10, 1]. They also allow engineers to repeat the same scenario under controlled conditions, including cases that would be dangerous or impractical to reproduce in reality. For example, wind or other environmental variation can be controlled so that different configurations can be compared more directly.

A key advantage of simulation is access to internal state. Unlike real world testing, simulation can provide direct observation of variables such as state estimates, planner outputs, controller commands, and ground truth references [11, 1]. These variables are important for understanding why the aircraft behaves in a particular way, yet they may be difficult to access during real world operation. For this reason, simulation is often one of the main environments where autonomous systems are developed and analysed.

For a simulation to be useful, engineers also need tools that help them see what happened. Visualisation is therefore a central part of simulation based analysis. In autonomous systems, behaviour emerges over time from sensor data, estimation, planning, decision logic, and control output. Raw numerical logs may contain the necessary information, but they are difficult to interpret on their own. Visualisation can reduce this burden by presenting several aspects of system state together in spatial and temporal context [7]. This is especially useful when internal states, external objectives, and obstacles need to be interpreted at the same time.

The importance of visualisation is directly linked to system autonomy, since control authority shifts from human operators or pilots towards autonomous algorithms. As a result, system behaviour becomes harder to interpret from external observations alone. In traditional piloted aircraft simulators, visualisation has primarily been designed to support piloting tasks, often with emphasis on visual realism, motion fidelity, and situational awareness [2]. Such environments can still support interpretation, but they are not mainly designed to expose internal autonomy processes. In contrast, simulation environments for autonomous systems are often used by engineers who need to inspect how the system estimates, reasons, and selects actions. This changes the visualisation requirements. Rather than focusing mainly on perceptual realism, visualisation must also prioritise transparency of internal system state and decision relevant information [10, 8]. The goal is not only to support operation, but also to support understanding.

Visualising internal system state can take many forms, from simple markers and overlays to more detailed representations of objects, trajectories, and sensor information. Common examples include displaying estimated vehicle position together with ground truth, or comparing planned and executed paths. It can also be useful to expose what a sensor observes and to visualise uncertainty in sensor readings [12, 13]. Such visualisations enable engineers to examine how sensor data and estimation errors propagate through the autonomy stack and how they influence planning and control decisions. Work on multi-robot worldview debugging also shows the value of comparing an autonomous system's internal representation with other available representations of the same situation [14].

As autonomy increases, human operators or pilots may no longer be directly involved in real-time decision making. Instead, humans often take supervisory or developmental roles, where the main need is to understand behaviour rather than directly control it. Visualisation can support this by communicating how the system represents its environment and what

decisions it is considering. This becomes especially important in safety critical systems where meaningful human oversight depends on some understanding of the system's internal processes [6].

Learning based methods can make this analysis even harder, since their decision processes are often less directly inspectable than explicit logic or model based components [15]. Visualisation can still support analysis, but only to the extent that relevant internal data is available. If the system does not expose useful internal state or decision related information, visualisation can only show the parts that are actually observable [16].

Simulation environments therefore provide a useful context for visualisation driven analysis. By offering access to internal system state, intermediate decision variables, and ground truth, they enable detailed examination of autonomous behaviour under controlled and repeatable conditions [1]. Visualisation is therefore a core part of simulation based development, especially when the goal is to understand why a system behaved in a certain way [11].

2.2 Autonomy Architectures and Decision Making

Autonomous systems can be built using many different decision making approaches. Some parts of an autonomy system may be based on explicit rules, state machines, Behaviour Trees, planners, or estimation models. Other parts may use data driven methods such as reinforcement learning, neural networks, or large language model based components. In practice, these approaches can also be combined within the same system, which means that different parts of the autonomy stack may expose different kinds of internal information.

For this thesis, the important distinction is not whether an approach is strictly model based or learning based, since these terms can mean different things in different technical fields. Instead, the focus is on how inspectable the decision process is during simulation. Some approaches expose explicit intermediate states, conditions, transitions, or planned actions. These can often be visualised more directly. Other approaches may expose outputs such as selected actions, probabilities, confidence scores, prompts, or logs, but the reasoning behind those outputs may be harder to interpret directly [15, 17].

This distinction matters for visualisation. A Behaviour Tree, for example, provides a structured execution state that can be shown through active nodes, return statuses, and transitions. A learned policy may still provide useful outputs, but additional methods are often needed to explain why a certain action was selected. Because the prototype in this thesis is based on Behaviour Trees, the implemented visualisation focuses on internal execution state that can be directly inspected and related to aircraft behaviour in the simulation.

2.3 Behaviour Trees in Autonomous Systems

Behaviour Trees are a hierarchical control architecture widely used in robotics and game AI to structure decision making and task execution. Colledanchise and Ögren introduce BTs as a way to organise behaviour through a tree structure with explicit execution rules [18, pp. 1–8]. BTs have increasingly been adopted in robotics due to their modularity, reactivity, and explicit execution semantics [19, 20].

A Behaviour Tree is composed of nodes organised in a tree structure. Execution proceeds through periodic *ticks* that propagate from the root to child nodes according to well-defined control flow rules. Nodes return one of three statuses: *Success*, *Failure*, or *Running*. Control flow nodes such as *Sequence* and *Fallback* (also called Selector) determine how execution

progresses depending on child node outcomes. A Sequence node ticks its children from left to right until one returns Failure or Running, whereas a Fallback node selects the first child that does not return Failure. Parallel nodes and decorator nodes provide additional structural flexibility [18, pp. 9–30].

A simple example is an aircraft mission tree where the root node first checks whether the vehicle is airborne. If not, the tree may execute a takeoff behaviour. Once airborne, a fallback subtree may check whether a no fly zone is nearby, and if so activate a keep out behaviour. If no such condition is triggered, execution may continue with nominal navigation towards the mission objective. In this way, the tree structure makes both the current behaviour and the logic behind transitions more explicit. Figure 2.1 shows a simplified version of this type of mission structure.

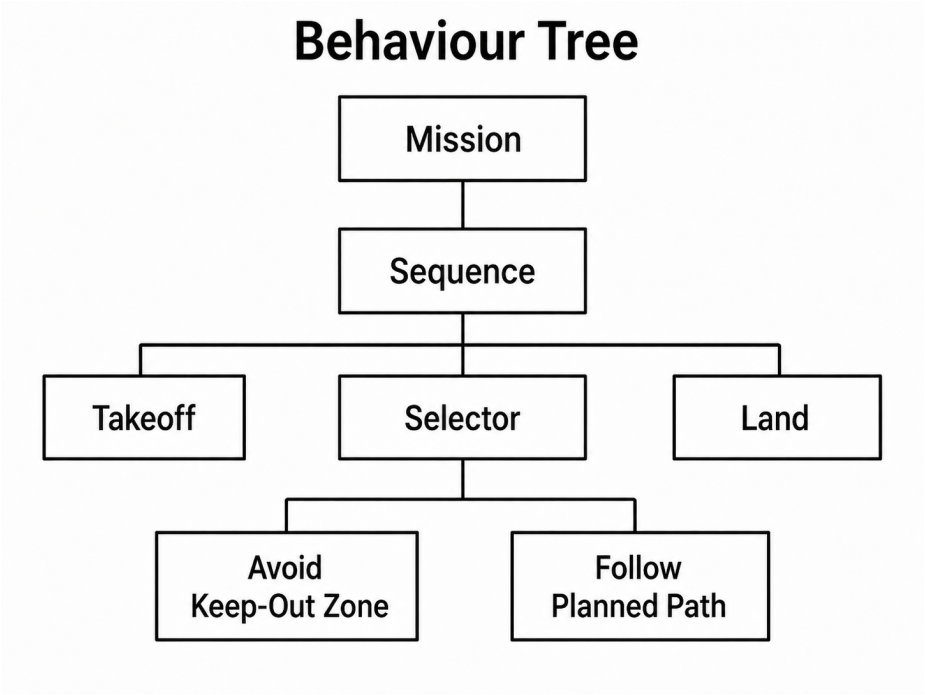


Figure 2.1: Simplified Behaviour Tree for an autonomous aircraft mission. The Sequence node represents the main mission order, where the aircraft first takes off, then selects between avoiding a keep-out zone or following the planned path, and finally lands.

Compared to traditional finite state machines, Behaviour Trees provide a more modular representation of decision logic. Rather than encoding all possible transitions between states, BTs separate the structure of control flow from the implementation of individual tasks. This separation allows behaviours to be extended or modified locally without restructuring the entire decision architecture. Marzinotto et al. [19] highlight that this property enables incremental development and simplifies the integration of new behaviours, which is particularly advantageous in complex robotic systems where mission logic evolves over time.

The formal properties of Behaviour Trees have been analysed in detail by Colledanchise and Ögren [21], who describe BTs as a control framework capable of representing hybrid decision systems while maintaining reactivity to environmental changes. Their work shows that the tree structure provides a clear representation of sequential and fallback behaviours while preserving responsiveness to new sensor information. In later work, Colledanchise and

Ögren describe the basic execution model of BTs, including ticks, node statuses, Sequence nodes, and Fallback nodes [18, pp. 9–30].

Behaviour Trees have also been applied in the context of unmanned aerial vehicles (UAVs), where mission execution often involves coordination between estimation, planning, and control subsystems. Ögren [22] describes how BTs can structure UAV mission logic by organising navigation tasks, contingency handling, and fallback behaviours within a unified hierarchical framework. This approach enables complex mission behaviour to be decomposed into smaller functional units while preserving a clear overall decision structure.

Behaviour Trees are not the only way to structure decision logic. Other approaches, such as finite state machines and related task level control structures, may also be used depending on the application. However, BTs are particularly relevant in this work because they combine modular task structure with explicit execution semantics, which makes them well suited for introspection and visualisation [19, 21, 20].

From the perspective of system analysis, Behaviour Trees expose a number of internal execution variables that can be informative when diagnosing autonomous behaviour. Examples include the currently active branch of the tree, node return statuses, failure propagation between nodes, and the frequency of transitions between subtrees. Because BT execution is explicitly structured and hierarchical, these variables provide a direct representation of the system's decision flow over time. As a result, BT-based autonomy architectures offer natural opportunities for introspection and visualisation of internal decision processes, particularly in simulation environments where execution traces can be inspected and replayed during debugging and analysis.

At the same time, as BTs grow in size and complexity, it can become more difficult to follow execution flow, inspect active subtrees, and understand why specific branches fail or become active. This makes visual support increasingly important when BT based autonomy is analysed in larger systems.

2.4 Simulation Ecosystems for Autonomous UAV Development

Development and evaluation of autonomous UAV systems are often done in simulation ecosystems that combine several tools rather than in one single simulator. A typical setup may include middleware for communication, a physics simulator, flight control software, and visualisation tools. In this thesis, this kind of ecosystem is not only an implementation detail. It also affects what information is available for analysis and how autonomous behaviour can be inspected [4, 23].

Middleware such as ROS 2 enables different parts of the system to communicate through messages. Sensors, state estimators, planners, controllers, and visualisation tools can publish and subscribe to data streams instead of being tightly connected through direct function calls [23, 4]. This is useful for analysis because internal state can often be recorded, replayed, and visualised. At the same time, it also means that information is spread across several asynchronous data streams. Messages may have different update rates, delays, and reference frames, which can make it difficult to understand the overall behaviour without suitable visualisation support [24, 25].

Physics based simulators such as Gazebo provide a controllable environment where vehicle dynamics, sensors, and environmental interactions can be tested before using physical hardware [10]. Simulation also makes it possible to repeat scenarios and inspect behaviour at

specific moments. This is important for autonomous systems, since relevant decisions and state transitions may happen quickly and may only become understandable when the run can be replayed or inspected afterwards.

Another useful aspect of simulation is access to ground truth. Internal estimates can be compared with simulator reference values, for example by comparing an estimated path with the actual simulated path. This can make errors such as drift, offsets, or frame problems easier to find [11]. However, this also depends on the data being shown in a way that engineers can interpret. A simulator view can show where the aircraft moved, but it does not by itself explain why the autonomy system selected a certain action or entered a certain state.

Autonomy and autopilot software also shape what can be visualised. Open source stacks such as PX4 and ArduPilot[1] are widely used in UAV development and can be connected to middleware such as ROS 2 [26]. These systems expose some internal variables, but not necessarily all information that is useful for understanding higher level autonomy. Visualisation tools must therefore either use the data that is already available, or add additional data streams for the specific analysis task.

Tools such as RViz,[2] Foxglove,[3] and Rerun[4] are useful for inspecting spatial data, messages, logs, and time dependent behaviour. Still, these tools are usually generic. They can show many kinds of data, but they do not automatically explain autonomous decision making. For this thesis, this distinction matters because the goal is not only to show the aircraft moving, but also to connect that motion to internal autonomy state, Behaviour Tree execution, selected internal values, and timing.

[1] ArduPilot documentation: `https://ardupilot.org`

[2] RViz documentation: `https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html`

[3] Foxglove documentation: `https://docs.foxglove.dev/docs`

[4] Rerun documentation: `https://rerun.io/docs/overview/what-is-rerun`

3 Related Work

This chapter reviews research relevant to understanding and visualising internal state in autonomous and robotic systems. The reviewed literature spans several related domains, including transparency in autonomous systems, visualisation for robotic debugging, runtime tracing and monitoring infrastructure, Behaviour Tree analysis, and visualisation of uncertainty. Together, these works provide conceptual and technical context for the present thesis.

3.1 Transparency and Explainability in Autonomous Systems

This line of research is important for the present thesis because it establishes why access to internal system processes matters when analysing autonomous behaviour. Transparency and explainability studies show that observable actions alone are often insufficient for understanding why a system behaved in a certain way, especially in safety critical settings. At the same time, much of this work is conceptual, explanation oriented, or focused on trust, accountability, and governance rather than on engineer facing visualisation during simulation based analysis. The present thesis builds on these ideas, but shifts the focus toward how internal autonomy state can be visually exposed in a way that supports interpretation during development and debugging.

Sparrow et al. [5] examine how autonomous systems should be evaluated beyond surface level task performance. Their work argues that judging autonomous behaviour solely based on outcomes can be misleading, particularly when the internal information available to the system differs from what a human observer assumes. The authors emphasise that evaluation must account for the system's internal state, perception limits, and optimisation criteria. In practice, this means that understanding autonomous behaviour requires insight into what the system knew at a given moment and how that information influenced its decision making process. Their contribution is primarily conceptual, establishing that meaningful assessment of autonomy requires access to internal reasoning rather than only observable actions. This supports the thesis focus on internal execution state, but does not by itself describe how such information should be visualised for engineers.

Wortham et al. [27] propose a taxonomy that categorises transparency mechanisms in robotic systems into perception transparency, reasoning transparency, and action transparency.

Perception transparency concerns what the system senses and how it interprets sensory input. Reasoning transparency concerns how internal representations are processed to produce decisions. Action transparency concerns how decisions are translated into observable behaviour. The taxonomy provides a structured framework for identifying which aspects of internal system state may need to be communicated in order to support understanding. While the work does not focus on simulation based debugging, it clarifies that transparency must address multiple layers of autonomy rather than only final actions.

Hayes and Shah [28] address explainability from the perspective of controller policy representation. They develop methods for translating internal decision policies into structured explanations that humans can interpret. In user studies, participants were better able to understand robot behaviour when provided with explanations derived from the system's internal policy model. The work focuses on generating discrete explanations rather than continuous visualisation, but it demonstrates that exposing internal control logic can improve interpretability. For this thesis, the relevance is the link between internal decision structure and human interpretation, while the difference is that the prototype explores coordinated visual views during simulation rather than generated policy explanations.

3.2 Visualisation for Robotic Debugging and System Analysis

This body of work is closely related to the present thesis because it examines how visualisation design affects developers' ability to analyse and diagnose robotic behaviour. These studies provide direct evidence that representation choices influence debugging performance, particularly when internal variables are presented in spatial and contextual relation to the system's task. However, most of the work is situated in general robotics or broader AI debugging contexts rather than in autonomous aircraft simulation. In addition, limited attention is given to higher level mission logic and decision flow, such as Behaviour Tree execution and state transitions, which are central in this thesis.

Ikeda and Szafir [8] investigate how different visualisation paradigms affect debugging performance in robotic systems. In their study, robotics practitioners were asked to diagnose predefined failure scenarios using three different interface styles: traditional 2D dashboards, spatially embedded 3D visualisations, and augmented reality representations. Performance was evaluated using diagnostic accuracy, task completion time, and subjective workload measures. The results showed that spatially embedded representations improved performance in failure cases involving trajectory deviations and spatial misalignment, suggesting that alignment between internal variables and physical context can support reasoning. The study compares entire interface paradigms rather than isolating specific variable encodings, but it provides empirical evidence that representation design directly influences diagnostic effectiveness.

Bae et al. [14] present MOSAIC Viewer, a visual analytics system for debugging worldview differences in cooperative autonomous multi-robot systems. Their work is relevant because it treats a robot's internal representation of the world as something that must be compared, inspected, and connected to scheduling decisions. The system links multiple views to help users understand where robots disagree and how these differences affect behaviour. This is close to the present thesis in its concern with internal autonomy state and coordinated views. The main difference is that MOSAIC focuses on multi-robot worldview desynchronisation, while this thesis focuses on autonomous aircraft simulation and the relation between Behaviour Tree execution, internal state, and spatial mission context.

Kolvig-Raun et al. [29] focus on debugging collaborative robotic systems and propose contextual visualisation techniques that integrate runtime information directly into task relevant views. Instead of presenting internal state in separate dashboards, their system embeds information such as grasp state, object pose, and robot configuration within the operational scene. Through evaluation with practitioners, they demonstrate that contextual embedding reduces the need for manual reconstruction of system state and supports faster identification of failure causes. Their findings indicate that proximity between internal state information and task context improves interpretability.

In a related contribution, Kolvig-Raun et al. [30] introduce EDDE, an event driven data exchange architecture designed to improve inspection of robotic execution. EDDE aligns internal state updates with discrete execution events rather than relying on fixed rate polling mechanisms. By structuring state information according to execution flow, the system improves temporal correspondence between internal changes and observable behaviour. Although the contribution is architectural rather than purely visual, the work demonstrates that temporal structuring of internal data plays a significant role in supporting debugging and analysis.

Within the broader field of visual analytics, Hohman et al. [16] provide a survey of systems designed to inspect complex AI models. The surveyed tools include visualisations of hidden layer activations, confidence measures, and training dynamics. The authors identify coordinated multiple views, interactive filtering, and temporal exploration as recurring design principles for analysing high dimensional internal state. While their focus is primarily on machine learning systems rather than robotics, the identified principles are applicable to structuring autonomy related data for human inspection.

3.3 Tracing and Monitoring in ROS Systems

Tracing and monitoring research is relevant to this thesis because it addresses how internal execution data can be captured, structured, and inspected in distributed robotic systems. This provides an important technical foundation for visualisation, since meaningful analysis depends on access to internal events, timing relationships, and causal chains across the autonomy stack. At the same time, these studies mainly focus on instrumentation, runtime verification, anomaly detection, and performance analysis rather than on how such information should be visually organised for engineers in simulation based debugging. The present thesis therefore relates to this work from a more human centred and visualisation oriented perspective.

Bédard et al. [24] introduce `ros2_tracing`, a framework for low overhead runtime tracing in ROS 2 systems. Their approach integrates middleware level instrumentation with operating system tracing to capture callback execution, message passing, and scheduling events in distributed robotic applications. Through case studies, the authors demonstrate how detailed trace data can reveal latency bottlenecks and execution anomalies that are difficult to detect through logs alone. The contribution primarily concerns data capture and instrumentation, providing a foundation for deeper inspection of internal execution behaviour.

Building on this work, Bédard et al. [25] propose a message flow analysis method that reconstructs causal chains across ROS 2 messages and callbacks. By deriving structured relationships between events, the system supports identification of complex timing dependencies and distributed execution paths. The work shows how raw trace data can be transformed into higher level representations suitable for analysis. However, the primary focus remains on execution tracing and causal reconstruction rather than on human centred visualisation

design.

Li et al. [31] present Autoware_Perf, a performance analysis tool targeting ROS 2 based autonomous driving pipelines. Their framework decomposes end to end latency into component level contributions and visualises execution graphs to locate bottlenecks. Similarly, Kuboichi et al. [32] introduce CARET, which analyses callback chains and communication paths to measure latency and execution ordering. Both systems emphasise the importance of internal timing variables and communication structure for diagnosing autonomy stack performance.

Across these tracing and monitoring efforts, substantial progress has been made in capturing and structuring internal execution data. The contributions primarily address instrumentation, anomaly detection, and performance analysis. The question of how engineers should visually organise and interpret these internal variables during simulation based debugging is less extensively addressed. This thesis uses that gap as part of its motivation: access to internal data is necessary, but engineers also need visual structures that make the data meaningful in relation to mission behaviour.

3.4 Behaviour Trees and Execution Analysis

This subsection is particularly important for the present thesis because Behaviour Trees form the main decision structure studied in the implemented prototype. Prior work on BTs has mainly focused on execution semantics, modularity, formal analysis, and practical deployment in robotic systems. This is highly relevant for the present thesis, since these same properties are also what make BTs attractive for visualisation: execution is hierarchical, explicit, and based on observable node states. At the same time, much of the existing work has focused on architecture, correctness, and control design rather than on how BT execution should be visually represented for engineers during simulation based analysis.

Marzinotto et al. [19] and Colledanchise and Ögren [21] present BTs as a modular and reactive alternative to more tightly coupled control structures. Their work explains how BTs separate control flow from the implementation of individual actions, which makes behaviours easier to extend and reorganise. Colledanchise and Ögren further introduce the basic BT concepts [18, pp. 1–8] and execution semantics [18, pp. 9–30] in book form. These studies provide an important theoretical basis for understanding why BTs are attractive in autonomous systems, but they do not primarily address how BT execution can be exposed visually during debugging or simulation based inspection.

Beyond formal analysis, BTs have also been studied in practical robotic applications. Ögren [22] shows how BTs can be used to structure UAV mission logic by organising nominal behaviour, contingency handling, and fallback mechanisms within a unified control framework. More broadly, Iovino et al. [20] survey the use of BTs across robotics and AI and show that they have become increasingly common in systems where modularity, reactivity, and maintainability are important concerns. Ghzouli et al. [33] further study how BT concepts are used in real robotic applications and compare them to more traditional behaviour modelling approaches. Together, these works show that BTs are not only a theoretical control formalism, but also a practical representation of decision logic in deployed robotic systems.

A related line of work has investigated verification and runtime analysis of BT based control logic. Colledanchise et al. [34] formalise the execution context of BTs for runtime verification, showing how BT behaviour can be monitored against desired properties during operation. This type of work is highly relevant because it highlights that BT execution provides a struc-

tured set of diagnostically meaningful signals, including active branches, status returns, and execution context. However, the main focus remains on formal monitoring and correctness rather than on human centred visualisation for engineering analysis.

Taken together, the BT literature explains why Behaviour Trees are useful as an execution model, and why their execution state can be monitored. Less attention is given to how engineers can inspect BT execution together with spatial mission context during simulation. This is where the present thesis positions its prototype: it treats the BT not only as a control structure, but also as a visual entry point into the autonomy system's reasoning.

3.5 Visualisation of Uncertainty

Uncertainty visualisation is relevant because autonomous systems often act on estimates rather than direct access to ground truth. Hullman [12] and Kay et al. [13] show that the way uncertainty is represented can affect how people interpret predictions, confidence, and system reliability. Daftry et al. [35] further show that internal confidence signals can be useful for detecting perception failures before they become visible as task failures. These studies are relevant because confidence and uncertainty can affect how system behaviour is interpreted. However, uncertainty visualisation was not evaluated as a separate feature in this thesis. It is instead treated as part of the broader problem of internal state visibility, where internal values may help engineers judge if the system acted on reliable information.

4 Method

This thesis studies how visualisation can support engineers when analysing autonomous aircraft behaviour in simulation. Since the focus is on interpretation and reasoning rather than numerical performance, the study follows a qualitative exploratory approach.

The work does not try to optimise aircraft performance or compare autonomy methods. Instead, it examines which internal states seem important to show, how they can be visualised, and whether such visualisation can support understanding during simulation based analysis.

4.1 Research Design

The thesis follows a qualitative exploratory approach. It combines pre-study interviews, prototype development, and a qualitative evaluation session in order to examine how visualisation can support interpretation of autonomous behaviour in simulation.

The overall process consisted of three phases:

1. A pre-study interview phase used to identify what information engineers need when analysing system behaviour, which internal signals they consider important, and what limitations exist in current workflows.
2. Design and implementation of a visualisation prototype in a simulation environment.
3. A qualitative evaluation based on scenario driven sessions where participants discussed internal behaviour and state transitions.

These phases also connect to the research questions. The pre-study mainly supported RQ1 by identifying what information engineers thought was important during analysis. The prototype development mainly supported RQ2 by exploring how that information could be structured and visualised. The evaluation then supported RQ3 by looking at how the resulting visualisation supported reasoning around anomalous behaviour.

4.2 Pre-study Interviews

The first phase of the study consisted of semi structured group interviews. The purpose was to identify what kinds of internal system information engineers rely on when analysing autonomous aircraft behaviour in simulation, what current tools support well, and where current visualisation approaches are insufficient.

This helped base the prototype design on real engineering tasks rather than only on assumptions from the literature. It also helped identify which aspects of autonomous system behaviour were seen as most difficult to understand in current workflows, which is in line with literature on observability and transparency in autonomous systems [27, 16].

The pre-study involved two interview groups with four participants in each group, resulting in a total of eight participants. The participants were employees at Saab and worked in projects related to mission supervision, mission autonomy, and nearby areas.

The interviews were carried out as semi structured group interviews. The group format was chosen because it enabled discussion between participants with related but not identical perspectives. It also made it possible to observe where participants agreed, built on each other's reasoning, or pointed to different challenges.

The interviews followed a semi structured format with open discussion. Rather than treating the interviews as fixed question and answer sessions, the questions were used as prompts to start discussion around engineering workflows, interpretation of autonomy, and visualisation needs.

Notes were taken during the sessions and later compiled into written summaries of the main thoughts, challenges, and ideas expressed by the participants.

The discussions were guided by a set of questions designed to capture both current practice and future needs. The original questions were formulated in Swedish (see Appendix A.1) but are presented here in translated form:

1. What are you working on today?
2. How long have you been working on it?
3. How do you interpret the concept of **autonomy** in the work you do?
4. When an autonomous behaviour looks wrong in simulation, how do you go about analysing it?
5. Which internal states or signals do you most often look at during troubleshooting?
6. Are there any internal states that are important but difficult to interpret in today's visualisations?
7. How important is it for you to see uncertainty or confidence in visualisations? How would you like that information to be presented?
8. If you could choose three things related to the visualisation of autonomous aircraft that should be visualised better, what would they be?
9. Looking ahead in the development of autonomous aircraft, is there any form of visualisation that could be interesting in the future?

The questions were chosen to connect the interview directly to the main topics identified in the literature. Questions about troubleshooting and internal signals were included because earlier work on transparency argues that observable behaviour alone is often not enough to understand autonomous systems [27]. Questions about uncertainty and confidence were included because uncertainty can affect how engineers interpret estimates, decisions, and possible failure causes [12, 13]. The final questions about missing or future visualisations were included to collect more open input from participants, since the goal of the pre-study was not only to confirm topics from the literature but also to find needs that were specific to their work context.

4.3 Prototype Development

The visualisation prototype was developed from two main sources: the literature review and the needs identified in the pre-study interviews. From the literature, the work used ideas related to transparency and observability, which motivated exposing internal execution state rather than only showing aircraft motion [27]. Work on uncertainty and interpretation also motivated making selected internal values visible, since engineers need to understand what information the system is using when making decisions [12].

From the interviews, the design was shaped by the need to understand internal decision logic in practice. Participants described a need to follow Behaviour Tree execution, inspect state transitions, and relate these to what the aircraft was doing in the environment. This led to a combination of spatial views and internal views, where aircraft motion, planned path, and no-fly zones could be interpreted together with Behaviour Tree execution and internal variables. This design choice links the pre-study to the prototype: the prototype does not try to visualise every possible signal, but focuses on the signals that participants connected to decision rationale and anomaly analysis.

The prototype was implemented in a simulation environment using tools commonly used in autonomous systems development. ROS 2 was used for data communication, Gazebo for simulation, PX4 for flight control, and RViz as the main visualisation interface. Behaviour Tree execution was handled through `py_trees`, with additional visualisation support provided through `py_trees_ros_viewer`.

Behaviour Trees were selected because the pre-study indicated that explicit rule based structures were easier to inspect than more opaque autonomy approaches. Since Behaviour Tree execution exposes active nodes, transitions, and return statuses, it was suitable for exploring how internal decision state could be visualised during simulation. This made it possible to show not only where execution currently was in the tree, but also how it changed over time and how those changes related to observed aircraft behaviour.

The prototype was intended to show both contextual simulation information and internal autonomy related information. The contextual part should help the user understand what the aircraft is doing in the simulated environment, for example in relation to its path, mission context, and no-fly zones. The internal part should help the user understand what the autonomy system is doing at the same time, especially in terms of Behaviour Tree execution, state changes, and internal values related to planning, estimation, and control.

The idea was to let several related views be used together rather than in isolation. Some views should support spatial understanding, such as where the aircraft is flying relative to the planned path and restricted areas. Other views should support understanding of internal decision making, such as which part of the Behaviour Tree is active and what information

is influencing that state. This was directly motivated by the interviews, where participants described a need to relate observed behaviour to internal execution when analysing a run.

4.4 Evaluation Method

The evaluation was designed as a qualitative follow-up interview session where participants watched simulation runs and discussed what they could understand from them. The focus was not mainly whether the aircraft completed a task, but whether participants could understand how and why the behaviour happened. This fits earlier work that argues that looking only at outcomes can hide important internal problems in autonomous systems [5, 27, 6].

The evaluation phase builds on the previous steps in the study. The pre-study interviews were used to identify what information engineers need during analysis and which aspects of autonomous behaviour are difficult to interpret. These findings guided the design of the visualisation prototype, especially the choice to combine spatial mission context with Behaviour Tree execution and selected internal values. The evaluation was then carried out to examine whether the prototype supported the types of reasoning and interpretation that were identified in the pre-study.

The evaluation involved three participants who had previously taken part in the pre-study interviews and who worked in areas related to mission autonomy or mission supervision. The intention was to include two participants from each interview group, focusing on individuals who had actively engaged in the earlier discussions and contributed with insights related to analysis and debugging of autonomous behaviour. In practice, three participants attended the evaluation session.

The same participant group was used because the evaluation was intended to build on the earlier discussions rather than test the prototype with completely new users. This made it possible to focus the session on whether the prototype addressed the needs and analysis problems that had already been identified.

4.4.1 Procedure

The evaluation was carried out as a group interview session structured around a set of simulation demonstrations. The session consisted of three main parts.

In the first part, participants were shown a simulation run where the aircraft behaved as expected and where the full visualisation was enabled. This scenario was based on keep out behaviour around a no fly zone and acted as a baseline. The aircraft performed a take-off, avoided the no fly zone, and then landed. The purpose of this part was mainly exploratory and to get participants familiar with the simulation setup and the different visualisation tools, as well as to discuss what information appeared useful when the system behaved as intended.

In the second part, participants were shown a different simulation run where the aircraft did not behave as expected. This anomalous run was based on the first one, but with an introduced offset in coordinates, which caused the Behaviour Tree not to enter the keep-out state and the aircraft to cross the no fly zone and not land. In this case, only the spatial visualisation was shown, meaning that participants could see the aircraft trajectory, planned path, and no fly zone, but not the Behaviour Tree viewer or Behaviour Tree heads-up display (BT HUD). Participants were then asked to discuss what they believed was wrong and what might explain the aircraft behaviour. The purpose of this step was to understand how participants reason about anomalous behaviour when only external system behaviour is visible,

which relates to research question 3.

In the third part, the same anomalous run was shown again, but now with the full visualisation enabled. As in the previous step, participants were first asked to reason about what was happening and what might be wrong, without being told the actual cause of the anomaly. After this discussion, the underlying issue in the simulation, which had been intentionally withheld, was explained. The purpose of this setup was to compare how participants' interpretations changed when additional internal information was available, and to support a discussion about whether the visualisation helped their reasoning.

After the three parts, the session ended with a more open discussion. This discussion concerned the tool more generally, what visual elements were useful or less useful, and what could be improved.

4.4.2 Interview Questions

The evaluation questions were adapted to the structure of the shown scenarios. For the nominal scenario, the questions focused mainly on what information participants considered important for understanding system behaviour and how useful the different visual elements appeared. For the anomalous runs, the questions focused more on diagnosis, interpretation of system state, and whether the added visualisation made the issue easier to understand.

The original questions were formulated in Swedish (see Appendix A.2) but are presented here in translated form:

1. Which information in this scenario is most important for understanding what the system is doing?
2. Which visual elements help you understand the behaviour?
3. Is there anything missing or unclear in the current visualisation?
4. Based on what you see, what do you think is wrong in this run?
5. What became clearer when the additional visualisation was shown?
6. Which visual elements contributed most to understanding or debugging the issue?
7. What aspects of the prototype were most useful overall?
8. What could be improved or added in future work?

As in the pre-study, the evaluation material consisted of written notes. Notes were taken during the session and later organised into a summary of the discussion.

The analysis was done by going through the written notes from the group discussions and the evaluation session, identifying recurring statements and observations, and grouping them into themes related to the research questions. This was a lightweight thematic summary rather than a full transcript-based thematic analysis, since the material consisted of notes rather than complete transcripts [36].

The analysis mainly focused on which internal states were treated as important, how Behaviour Tree transitions were discussed, what parts of the visualisation seemed to support or hinder understanding, and how interpretations differed between the limited and full visualisation conditions.

4.5 Ethical Considerations

Participation in the pre-study and evaluation sessions was voluntary. Participants were involved because their work was relevant to the thesis topic, and the discussions focused on engineering workflows, interpretation of autonomous behaviour, and visualisation needs. No sensitive operational scenarios were analysed in the thesis, and the work was limited to simulation based development contexts.

There is also a broader ethical relevance to the thesis topic itself. Autonomous systems in safety critical domains require tools that support transparency and meaningful human oversight during development. While this thesis does not claim to solve those wider issues, the methodological focus on visualising internal state is partly motivated by the need to make autonomous behaviour more understandable and open to scrutiny [6, 17].

5 Result

This chapter presents the results from the study. It is split into three parts. First come the pre-study interview findings. After that the prototype is described, with focus on how the interview findings influenced the design choices. Last comes the results from the evaluation session based on the three simulation runs and the discussion that followed.

5.1 Findings from the Pre-Study Interviews

The pre-study findings are based on notes from two group interviews. Since the interviews were not recorded or transcribed, the results are presented as group level summaries rather than exact quotations. The findings are grouped into themes that appeared in the interview notes and that later influenced the prototype design. Table 5.1 summarises these themes and shows how each theme affected the prototype design.

The two interview groups worked with different kinds of autonomy. The first interview focused mainly on mission autonomy, human-autonomy teaming, dynamic mission planning, and large language model based support for high level mission decisions. The second interview included work related to reinforcement learning, Behaviour Trees, mission control, and more classical agent techniques. Even though the technical approaches differed, both interviews pointed to the same general problem: when an autonomous system behaves unexpectedly, engineers need to understand why it behaved that way and not only whether the mission succeeded or failed.

A first theme was that different autonomy approaches provide different levels of observability. In the first interview, LLM based autonomy was discussed in terms of prompts, sensor inputs, output logs, and reasoning traces. Analysis was described as looking at what information was given to the model and what response it produced. In the second interview, reinforcement learning was described more in terms of training logs, many repeated runs, benchmark outcomes, and whether the agent succeeded or failed. In contrast, rule based approaches and Behaviour Trees were described as easier to inspect, since they expose a more explicit decision structure.

Table 5.1: Main themes identified in the pre-study interviews.

Theme	**Support from interview material**	**Design implication**
Different observability between autonomy approaches	Interview 1 discussed large language model based autonomy, reasoning traces, prompts, and input/output logs. Interview 2 contrasted reinforcement learning and Behaviour Trees, and described rule based or Behaviour Tree based systems as easier to inspect.	Focus the prototype on a Behaviour Tree based case where internal execution can be shown directly.
Need to understand decision rationale	Interview 1 mentioned visualising key decisions, alternative decisions, and what information influenced a decision. Interview 2 also raised key decisions and why states were entered or why something did not succeed.	Show active BT state, state transitions, and internal values related to the current behaviour.
Replay and temporal inspection	Interview 1 mentioned replaying a specific decision in time to see what happened and why. Interview 2 noted that earlier decisions can affect the outcome much later in a run.	Support analysis over time and motivate replay or stepwise inspection.
Relation between high level plan and execution	Interview 1 discussed agreement between high level plan and lower level execution. Interview 2 mentioned mission control, planning, manoeuvring, and restricted areas.	Combine spatial mission context with internal execution information.
Task dependent information needs	Interview 1 separated high level mission planning from lower level sensor information. Interview 2 warned against showing too much information and suggested focusing on tree location, leaves, conditions, and paths through the tree.	Avoid treating all internal variables as equally important and motivate a selective BT HUD.

A second theme was the need to understand decision rationale. In the first interview, participants mentioned visualising key decisions, alternative decisions, and the information that influenced a decision. They also discussed the need to show agreement between high level mission plans and lower level execution. In the second interview, similar issues appeared around why the system entered a certain state, why something failed, and how different possible decisions could be compared. This supported the decision to focus the prototype on internal decision state rather than only external aircraft motion.

Replay and temporal analysis also appeared in both interviews. In the first interview, participants described the need to replay a specific decision in time and then see what happened and why. In the second interview, participants pointed out that an outcome may depend on a decision made much earlier in the run. One example discussed was that something done a minute earlier could be the reason why an agent later succeeded. This made replay and inspection over time important for the design, especially for Behaviour Tree execution where the active node and execution path can change during a run.

The Behaviour Tree discussion in the second interview gave more concrete input for the prototype. Participants mentioned that it would be useful to see where execution currently is in the tree, which node is active, how conditions and leaves are evaluated, how the system moves between states, and what path through the tree has been taken. They also mentioned scenarios involving manoeuvring, planning, and areas that should not be entered. These points directly motivated showing BT execution together with spatial information such as planned path, flown path, and no fly zones.

Another theme was that information needs depend on the task and the user. The first interview made a distinction between high level mission planning and lower level sensor information. The second interview also warned against showing too much information and suggested that the visualisation should focus on what is currently useful, such as the current node, conditions, and relevant decision information. This later influenced the BT HUD design, but also showed a limitation: simply exposing more internal values is not enough if the information is not structured around the current analysis task.

Taken together, the pre-study showed that observability, traceability, replay, and decision rationale were important parts of simulation based analysis. The interview material also pointed toward Behaviour Trees as a suitable focus for the prototype. Compared with large language model based or reinforcement learning based approaches discussed in the interviews, Behaviour Trees provided a more explicit structure for showing execution state, transitions, and decision paths.

5.2 Prototype Implementation

The prototype was designed based on the findings from the pre-study together with ideas from the literature. The main goal was to make internal decision making more visible during simulation, especially how and why the system transitions between different states. The pre-study showed a need to understand decision rationale, replay behaviour over time, and relate internal execution to spatial behaviour. These points guided the overall structure of the prototype.

A central design decision was to focus on Behaviour Trees (BTs), since the interviews indicated that rule based approaches were easier to inspect compared to reinforcement learning or large language model based systems. This made it possible to visualise execution explicitly, including which node is active, how execution moves through the tree, and how decisions

are made over time.

The prototype combines three main parts: a spatial view, a Behaviour Tree viewer, and a Behaviour Tree heads-up display (BT HUD). These are intended to be used together rather than separately.

The spatial view, shown in Figure 5.1, displays the aircraft, the planned trajectory, the executed path, and the no fly zone in RViz. This view mainly supports understanding of what the aircraft is doing in the environment. It makes it possible to compare intended and actual motion and to see when constraints such as no fly zones are violated. This part of the design is based on the pre-study, where participants described that they first observe the overall behaviour before looking into internal details.

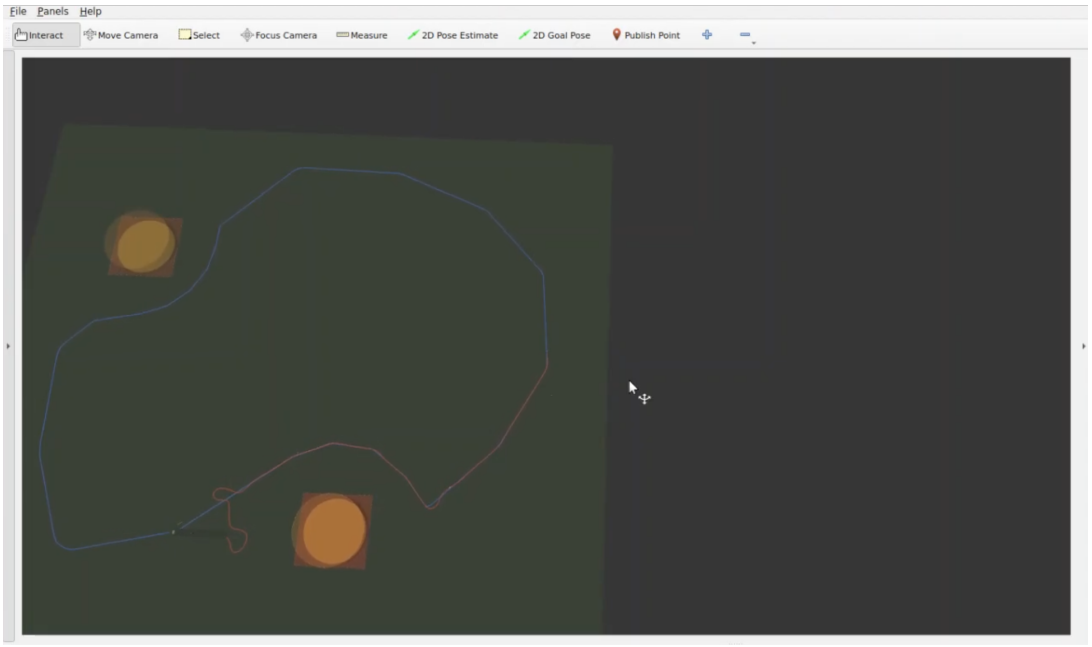


Figure 5.1: Spatial view from the prototype in RViz. The figure shows the aircraft path, flown path, and no fly zones used in the first keep-out scenario.

On the right side of the full interface, shown in Figure 5.2, the Behaviour Tree HUD (BT HUD) presents additional internal information related to the current execution. This includes selected variables from planning, estimation, and control, as well as higher level explanations of what the system is doing. The idea behind this part comes from the pre-study, where participants asked for insight into what information the system is using and what influences a decision. The BT HUD therefore complements the tree structure by adding more detailed context.

The three views are intended to be used together. A typical interaction is to first observe the aircraft behaviour in the spatial view, then look at the Behaviour Tree to see which state the system is in, and finally use the BT HUD to understand why that state was entered. This reflects the workflow described in the pre-study, where analysis often moves from observing behaviour to explaining it.

Overall, the prototype is structured around linking internal execution to external behaviour. The spatial view shows what happens, while the Behaviour Tree viewer and BT HUD aim to

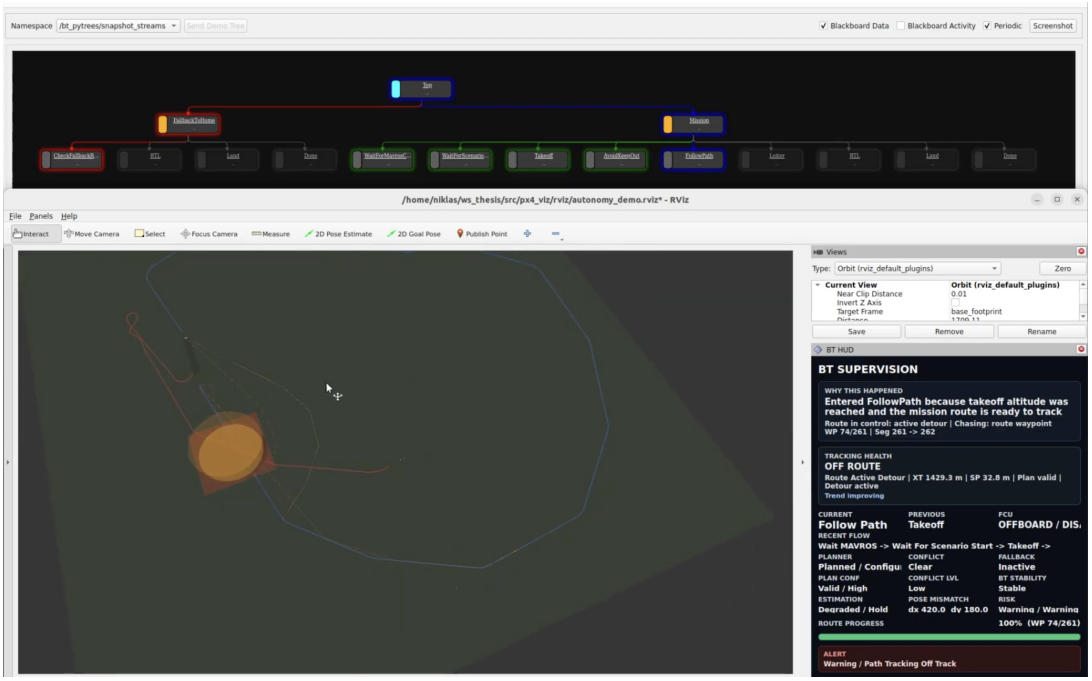


Figure 5.2: Full prototype view used in the third evaluation scenario. The upper part shows Behaviour Tree execution, the centre shows the spatial scene in RViz, and the right side shows the BT HUD with selected internal state related information.

show why it happens. This combination was chosen based on the pre-study findings, where participants described a need to relate visible aircraft behaviour to internal decision logic.

5.3 Evaluation Results

The evaluation results are based on notes from the group interview session. Since the session was not recorded, the results are presented as group-level summaries rather than exact quotations. The evaluation was structured around three simulation runs. The first run showed nominal keep-out behaviour with full visualisation enabled. The second run showed anomalous behaviour with only limited spatial visualisation. The third run showed the same anomalous behaviour again, but now with the full visualisation enabled. After the scenarios, the participants discussed the tool more generally and suggested possible improvements.

5.3.1 Scenario 1: Nominal Behaviour with Full Visualisation

The first scenario was used as a baseline. In this run, the aircraft behaved according to the intended keep-out logic. It took off, followed the planned path, avoided the no fly zones, and then landed. The purpose of this scenario was to let participants become familiar with the simulation setup, the Behaviour Tree viewer, and the Behaviour Tree heads-up display (BT HUD), and to discuss what information was useful when the system behaved as intended.

A simplified sketch of the scenario is shown in Figure 5.3. The red line represents the planned path, while the black line shows the actual path taken by the aircraft after replanning. The square areas mark the no fly zones that the aircraft is expected to avoid. In this nominal case, the aircraft follows the path around these areas and completes the mission.
The full interface used during the evaluation is shown in Figure 5.2. This includes the spatial view, the Behaviour Tree viewer, and the BT HUD, which were all visible to the participants

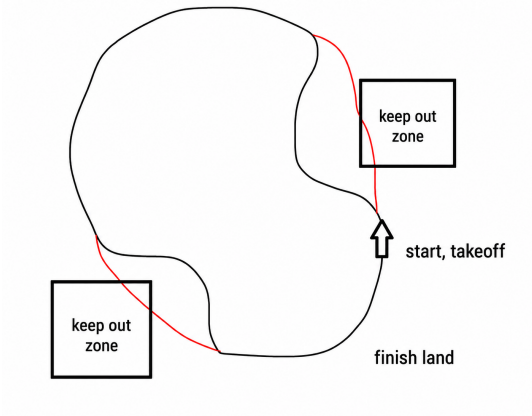


Figure 5.3: Simplified sketch of the nominal scenario. The planned path (red) and the actual path (black) are shown together with the no fly zones.

during this scenario.

In the evaluation discussion, participants described the Behaviour Tree related views as useful, especially the Behaviour Tree viewer and the BT HUD. The Behaviour Tree viewer was described as helpful because it made it easier to follow how execution moved through the tree and to see where the system currently was. Participants also mentioned backtracking through the tree and colour indications as useful for following progression. At the same time, the lower part of the interface, where more detailed information was shown, was seen as interesting but also somewhat heavy to interpret due to the amount of information.

The evaluation notes also show that the combination of views mattered. Participants did not only look at the Behaviour Tree by itself, but connected it to what the aircraft was doing spatially. The spatial path, no fly zones, and internal views together helped frame what the system was doing and what it was expected to do. The group also commented that the structure of the information was not always fully clear and that some parts of the BT HUD seemed more useful than others.

Another point from the discussion was that the usefulness of the visualisation may depend on the underlying autonomy architecture. Participants discussed that the current approach was closely tied to Behaviour Trees, but that a more dynamic or configurable interface could make the same visual environment useful for other types of systems. This was especially connected to the BT HUD, where participants suggested that being able to choose what to display would make the tool more useful for different users and tasks.

Overall, the first scenario showed that the tool was useful for understanding normal execution, especially through the Behaviour Tree related views. At the same time, the participants' responses also showed that the amount and structure of information can become a problem, and that some parts of the interface may need to be simplified or made more focused.

5.3.2 Scenario 2: Anomalous Behaviour Without Additional Visualisation

In the second scenario, the participants were shown a run where the aircraft did not behave as expected. The aircraft crossed the no fly zone instead of avoiding it, which made the behaviour unexpected in relation to the intended keep-out behaviour. This run was shown with limited visual support, meaning that participants could see the aircraft motion, planned

path, flown path, and no fly zone, but not the Behaviour Tree viewer or BT HUD.

Figure 5.4 shows the anomalous scenario with limited visual support. The purpose of this part was to see what participants could understand from aircraft behaviour alone, before internal decision related views were added.

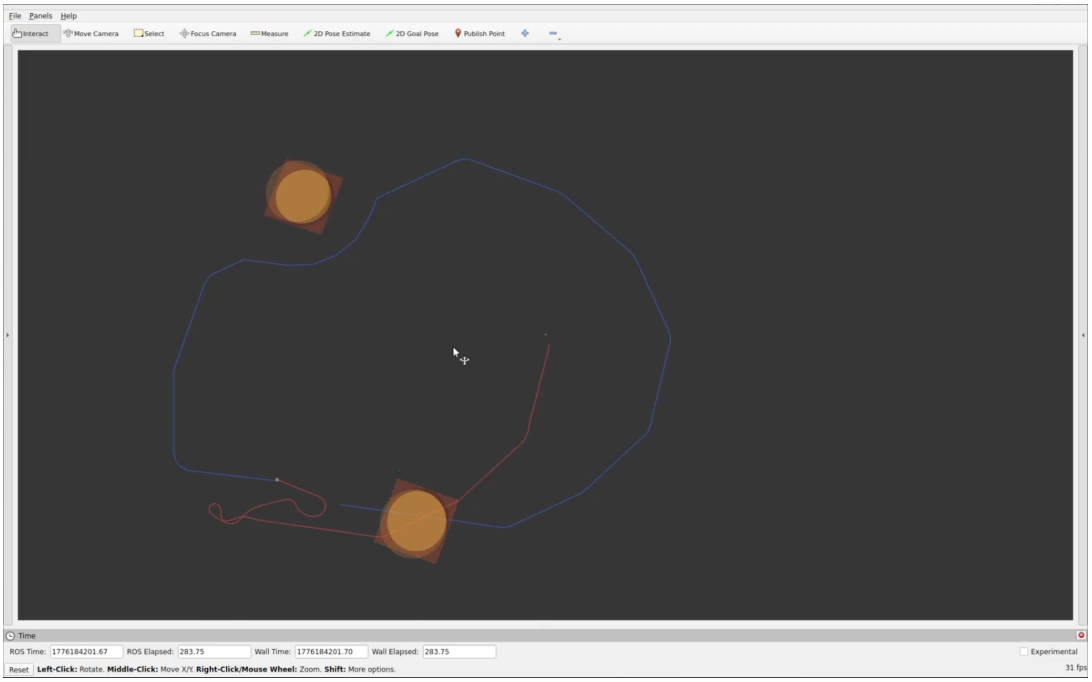


Figure 5.4: Anomalous scenario shown with limited visual support. The aircraft behaviour can be observed in the scene, but the additional internal decision related views are not available in the same way as in the full prototype view.

In the evaluation discussion, participants quickly identified that something was wrong with the aircraft behaviour. They observed that the aircraft flew over the no fly zone and that it was difficult to understand its direction. The first explanations focused on possible coordinate errors, directional errors, or that the system might be in the wrong internal state. However, the notes also show that these explanations were treated as guesses rather than confirmed causes.

The discussion also showed what information the participants missed. They wanted to know which state the system was currently in, whether it was in the wrong state, and what had happened in the Behaviour Tree before the aircraft crossed the no fly zone. Participants also raised questions about whether the wrong action had been executed in the tree and how the Behaviour Tree was connected to the aircraft behaviour.

This scenario therefore made the limitation of the reduced view visible. Without the Behaviour Tree viewer and BT HUD, the participants could suggest plausible explanations based on what they saw, but they could not validate them. The observed aircraft motion was enough to see that something was wrong, but not enough to explain the cause with confidence.

5.3.3 Scenario 3: Anomalous Behaviour With Full Visualisation

In the third scenario, the same anomalous run was shown again, but now with the full visualisation enabled, as shown earlier in Figure 5.2. The participants were again asked to explain what they believed was wrong, before the actual cause of the anomaly was explained. The purpose was to compare how their reasoning changed when internal decision related information was available.

In the evaluation discussion, participants were able to reason more clearly compared to the previous scenario. The earlier guesses related to coordinates and direction were still present, and the notes show that these guesses were partly correct. However, the participants still did not fully isolate the exact cause before it was explained. They identified that something was wrong with coordinates or direction, but could not clearly separate whether the issue came from planning, estimation, or another internal component.

The main difference compared to the previous scenario was that the participants could now check their assumptions against the visualised internal state. The Behaviour Tree viewer made it possible to see which state the system was in and how execution moved through the tree, while the BT HUD provided additional internal values. Together with the spatial view, this gave more context for relating aircraft behaviour to internal decision making.

According to the evaluation notes, the participants described the combination of views as the most important part. They pointed out that a single visualisation was not enough on its own, but that the Behaviour Tree, BT HUD, and spatial view together made it easier to understand what was happening. The visualisation did not directly give them the answer, but it made it possible to validate or reject their earlier guesses.

The discussion also highlighted that experienced engineers often have standard guesses about what might be wrong, such as coordinate errors or incorrect state transitions. However, without internal visualisation these remain guesses. With the full visualisation, the participants could connect their assumptions to the actual system behaviour and internal state, which made the reasoning more grounded.

Some limitations were also discussed. Participants suggested that better temporal support, such as stepping through the execution over time, would improve the analysis further. They also mentioned that different parts of the Behaviour Tree or internal variables might be more relevant depending on the situation, which again points toward a need for more dynamic visualisation.

A final reflection from the discussion was that an agent can sometimes do the right thing for the wrong reason, or behave incorrectly for reasons that are internally consistent given faulty information. This connects to the main motivation of the thesis, since understanding autonomous behaviour requires access not only to what happened, but also to the internal state and the decision process behind it.

5.3.4 General Reflections After the Scenarios

After the three scenarios, the participants discussed the tool and the general approach in a more open way. The following points are based on the group discussion notes rather than exact quotations.

A clear theme in the discussion was that the combination of multiple views was more useful than any single visual element on its own. Participants described that they used the spatial

view, the Behaviour Tree viewer, and the BT HUD together when reasoning about the system. The spatial view was generally seen as clear and easy to understand, while the Behaviour Tree viewer was useful for following execution flow. The BT HUD, on the other hand, was described as the part that contained the most information and therefore required more effort to interpret.

The discussion also showed that the issue with the BT HUD was not that the information was irrelevant, but that it was not equally useful in all situations. Participants described that engineers typically focus on a subset of variables depending on the task, rather than inspecting everything at once. Based on this, they suggested that the BT HUD should be more dynamic, so that different variables can be selected depending on whether the focus is on Behaviour Tree execution, planning, estimation, or another part of the system.

Another recurring point in the discussion was the need for temporal support. Participants described replay and stepwise inspection as important for analysis. They wanted to be able to move through a simulation run and inspect how the Behaviour Tree, internal variables, and aircraft motion changed over time. This was connected to the idea that understanding often depends on seeing what happened just before and after a specific decision.

The anomalous scenario was also discussed further. Participants suggested different possible reasons for why the coordinates could have been wrong, including offsets, incorrect inputs, or mismatches earlier in the execution. This showed that even with full visualisation, there can be multiple plausible explanations. However, the visualisation made it easier to reason about these possibilities and compare them to the internal system state.

Participants also discussed how the interface could be extended. One suggestion was to make the visualisation more flexible so that different views or variables could be selected depending on the user or the underlying autonomy system. This was connected to a broader discussion about whether the same approach could be used for systems that are not based on Behaviour Trees, such as more dynamic or learning-based approaches.

Another reflection from the discussion was that the prototype helped frame what the system was doing, but that it was sometimes unclear what the current task or objective of the agent was. Participants suggested that showing what the agent is currently trying to achieve could be just as important as showing where execution is in the Behaviour Tree.

6 Discussion

This chapter analyses the main findings from the study and discusses what they imply for visualising internal autonomy processes in simulation. The discussion is structured around the most important topics rather than following the same order as the previous chapters. These topics include the value of combining several views, the role of Behaviour Trees in interpretation, validation of explanations, temporal analysis, information density, methodological limitations, wider context, and future work.

6.1 Linked Views and Behaviour Tree

One clear thing in the results is that the visualisation did not help because of one single view. What seemed to matter was the combination of views. This came up both in the pre-study and in the evaluation. The participants used the Behaviour Tree viewer, the Behaviour Tree heads-up display (BT HUD), and the spatial context around the aircraft together. The value came from reading these views together, not from one isolated element. This matches earlier work on transparency and debugging, where understanding often depends on relating internal system information to external behaviour [27, 8, 29].

The Behaviour Tree viewer had a central role in this. The pre-study showed that participants wanted to understand where execution was in the decision structure, why a state became active, and how execution moved between branches. The evaluation showed the same kind of need in practice. When the Behaviour Tree viewer was available, the participants could relate aircraft behaviour to the current execution state. This made the Behaviour Tree more than just a separate technical representation. It became a way of connecting internal decision logic to what was happening in the simulation.

The spatial view was still important, but it served another role. It helped frame what the aircraft was doing in relation to the planned path and the no fly zone. The Behaviour Tree viewer helped explain what the autonomy system was doing internally. The BT HUD then added more detailed values that could support interpretation. Together these views gave a broader picture of the run than any of them could provide alone.

This also shows why the prototype focused on Behaviour Trees. In the interviews, rule based and Behaviour Tree based systems were described as easier to inspect than more opaque approaches. This does not mean that Behaviour Trees are always better as an autonomy architecture, but they are useful for this kind of visualisation because they expose a structure that can be followed over time. Active nodes, transitions, and return statuses can be shown directly, which makes them suitable for analysis in a simulation environment.

6.2 Interpretation and Validation of Explanations

Another important result was that the prototype seemed to support validation more than direct explanation. In the anomalous run without the added internal views, the participants could still suggest possible causes from the spatial view alone. They mentioned coordinate errors, direction errors, or that the system might be in the wrong Behaviour Tree state. These explanations were based on what they could infer from the aircraft motion and on their previous experience with common failure types, but they could not be checked against the internal state in that view.

When the same run was shown again with the full visualisation, the participants could compare those possible explanations with the Behaviour Tree execution and the values shown in the BT HUD. The visualisation did not simply give them the answer. It made it easier to judge whether an explanation was reasonable. This distinction is important for RQ3. The study does not show that the prototype made participants diagnose the problem perfectly or independently. What it does show is that the visualisation helped them reason in a more grounded way.

This is also connected to the idea that an autonomous system can do the wrong thing for reasons that are internally consistent. If the system acts on incorrect position information, a planner or Behaviour Tree may behave consistently according to that internal state, even if the external behaviour looks wrong. This means that understanding the behaviour requires access to both what happened externally and what information the system acted on internally.

The study therefore supports a more limited but useful claim. The prototype did not remove the need for engineering judgement. Participants still had to interpret the situation and discuss possible causes. However, the added visualisation gave them more support for checking their interpretations and relating visible behaviour to internal execution.

6.3 Temporal Analysis and Replay

The pre-study also showed how engineers often analyse a run in practice. A run is often watched to the end first just to see the outcome. After that, it is replayed and inspected more closely to look at key decisions and events that may have shaped the outcome. This appeared again in the evaluation, where participants repeatedly returned to the need for replay and stepwise inspection.

This means that the visualisation needs to support more than live viewing. Replay and later analysis matter too. To understand a state transition, participants wanted to inspect what happened just before and just after it while seeing the Behaviour Tree, internal values, and aircraft motion together. This is especially relevant for Behaviour Trees, since execution changes over time and a decision made earlier in a run can affect behaviour later.

The prototype touched this issue, but only to a limited extent. The Behaviour Tree visualisation made it possible to follow execution during a run, but the evaluation also suggested

that stronger support for time based inspection would be useful. For example, being able to step through the run and keep the spatial view, Behaviour Tree execution, and BT HUD synchronised would make it easier to analyse why a particular transition happened.

This is one of the clearer future directions from the study. The question is not only which internal information should be shown, but also how that information can be inspected over time.

6.4 Information Density and Task Dependent Views

The results also show that visibility is not the same thing as clarity. The clutter problem did not mainly come from the spatial view or the Behaviour Tree viewer. It came more from the BT HUD. The issue was not that the information was useless, but that some variables were more relevant than others depending on the task and the person using the tool.

This was already visible in the pre-study. Participants described that engineers usually focus on the variables that matter for the part of the system they are working with, instead of trying to inspect everything at once. Someone working with Behaviour Tree execution may care most about active nodes and transitions. Someone working with estimation or planning may need different values. The evaluation confirmed this issue when participants described the BT HUD as interesting but information heavy.

Because of this, the design problem is not only about showing more state. It is also about showing the right state. A more useful version of the BT HUD would probably need to be more dynamic, so that users can select or filter information depending on the current analysis task. This could also make the tool more useful for different users, since the same run may be interpreted from different engineering perspectives.

This also limits how the current prototype should be understood. It should not be treated as a finished interface. It is better seen as an exploratory implementation that shows the value of combining internal and spatial views, while also showing that too much internal information can become difficult to use.

6.5 Scope and Methodological Limitations

The pre-study was useful because it gave the prototype a clearer direction. Without it, the design would have relied much more on assumptions from the literature and my own ideas about what should be visualised. The interviews helped narrow the scope and pushed the work more toward Behaviour Trees, state transitions, and decision related information instead of trying to visualise everything at once.

There are still limits in what can be claimed from the pre-study. The participants came from one industrial context and the sample was small. The interviews were also done in groups and documented through notes instead of recordings or transcripts. The group format helped the discussion, since participants could build on each other's answers and bring in examples from related work areas. At the same time, it makes the material less precise. It is hard to know which ideas were strongly shared and which ones were shaped by the discussion in the room. Since the interviews were not transcribed, the analysis depends quite a lot on what was written down during and after the sessions.

The implementation also affects the scope of the findings. The prototype was actually built and used in a simulation environment with tools such as ROS 2, Gazebo, PX4, RViz, and

`py_trees`. This made the work more concrete and made the evaluation more meaningful, since the participants were shown something closer to a real engineering environment than a mock-up. At the same time, the prototype focuses on one type of autonomy architecture and one kind of scenario. It also combines several visual elements at once, which makes it harder to say exactly which part contributed most to interpretation.

The evaluation design was useful for creating discussion around the prototype, but it should not be treated as a controlled comparison. One useful part was that the same anomalous run was shown first without added internal views and then again with the full visualisation. This made it possible to discuss what changed when the additional information was available. However, because the participants saw the same underlying case twice, the second viewing was affected by familiarity with the scenario. A stronger study would need more scenarios, and ideally some form of counterbalancing, so that the effect of visualisation could be separated more clearly from the effect of repetition.

The group format was also both useful and problematic. It gave richer reflections, especially after the scenarios, and it allowed participants to respond to each other's reasoning. That was useful for this thesis, since the study is about interpretation and much of that came out through discussion. On the other hand, group interviews make it harder to judge individual understanding. Some participants may have been influenced by others rather than reaching the same conclusion on their own.

Another issue is that the final evaluation group was smaller than planned. The original plan was to include four participants, two from each pre-study group, but one invited participant was unable to attend the evaluation session. Since the study was already small, that reduction matters. With so few participants, each response carries more weight and the results become more sensitive to the specific people in the room.

The evaluation also stayed close to one kind of behaviour, namely keep out around a no fly zone. That was useful because the case is easy to understand and because it created a clear link between internal state and spatial context. At the same time, it makes the evaluation narrow. It is not clear that the same visualisation would work equally well for other anomalies, more complex missions, or autonomy architectures where the decision process is not represented as explicit states and transitions.

Replicability in this thesis is therefore limited rather than strong. The overall process is described well enough that someone else could follow the same structure: carry out a pre-study, implement a visualisation prototype, and evaluate it through scenario based interviews. Exact replication would still be difficult. The industrial context, simulation setup, prototype design, and participant background all affect the outcome. Another study would probably resemble this one rather than reproduce it exactly.

Reliability is also limited. If the study were repeated with another group of engineers, or even with the same participants at another time, some similar themes might appear again. For example, the need to understand why a decision was made, or the usefulness of relating Behaviour Tree execution to mission context, may come back. But the exact comments and emphasis would likely differ. This is partly because the study is qualitative, partly because the participant group is small, and partly because the material depends on group discussion and note taking rather than fixed measurements.

Validity is more mixed. On one hand, the study has some practical validity because the participants were relevant to the problem domain and because the prototype was shown in a realistic simulation context. The questions discussed in the thesis are also closely tied

to real analysis work. On the other hand, the study does not directly measure debugging performance or diagnostic accuracy in a strict sense. It mainly captures perceived usefulness, observed reasoning, and discussion about what became clearer when visualisation was added. Because of that, the thesis can say something about interpretability and analytical support, but less about measurable improvement.

There is also a validity issue in the relation between the research questions and the collected material. The first two questions are handled more directly, since the interviews and evaluation both produced material about what information is useful and how it might be structured. The third question is harder. The study gives some support for the idea that the visualisation improved interpretation, but the basis for saying that is still limited. So the thesis can give a cautious answer to RQ3, but not a strong one.

6.6 The Work in a Wider Context

Even though this thesis is limited to simulation based development, it still connects to wider ethical and societal questions. Autonomous aircraft are part of a safety critical area. In such systems, it matters whether engineers can understand why a system behaves the way it does. If the internal reasoning stays opaque, failures become harder to analyse, responsibility becomes harder to place, and trust may rest on too little understanding [6, 17].

This thesis can be seen as a small contribution toward increasing transparency during development. It does not solve transparency in any broad sense, and it does not make autonomous behaviour fully explainable, but it does explore one practical way of exposing internal state and decision related information in a more structured form.

At the same time, it is easy to overestimate what visualisation can do. A well designed interface may help developers inspect and reason about behaviour, but it can also create a false sense of understanding. A clean visualisation does not guarantee that the shown information is complete or that the internal logic is correct. This matters especially in simulation, where access to internal state and ground truth is much better than in real deployment. What works well in simulation may be harder to expose in real operation.

There is also a practical engineering side to this. Better visualisation support may reduce some of the guesswork that often appears in debugging autonomous systems. It may also help communication between engineers working with different parts of the stack and shorten the time between noticing a problem and forming a plausible explanation. That is not a dramatic theoretical contribution, but it can still matter in practice.

The thesis also points toward a broader issue for future autonomy systems. Some participants raised the question of what happens when the underlying decision system is changed from a Behaviour Tree to an approach where the decision process is less explicitly represented, such as a learned policy or another data driven method. As systems become less transparent internally, the need for meaningful analysis tools may become even greater.

6.7 Future Work

There are several directions for future work. One is to broaden the evaluation with more scenarios, not only keep out around a no fly zone but also other anomalies that may be easier or harder to interpret. A larger and more structured evaluation would also be useful. That could include more participants from different work areas, not only mission autonomy. The current study gives useful indications of what may be important to visualise, but not enough

to support broader claims.

A second direction is to improve the prototype itself. The current prototype is fairly basic, has limited visuals, and may miss information that turns out to be important in other analysis situations. The interviews also showed that the BT HUD was dense and mentally tiring to inspect. Future work could therefore focus on that part, remove redundant information, and structure it better. There is also room to improve the map view where the aircraft moves in the scenario.

Another direction is to improve temporal support in the simulation. RViz does not support this as well as might be needed, while other tools such as Rerun may potentially offer better support for replay and synchronised inspection. The Behaviour Tree visualisation already supports stepping through time to some extent, so part of the future work may be about refining the current tool chain or moving toward tools that better support replay and synchronised inspection. This seems important, since replay was repeatedly described as necessary for proper analysis.

There is also room to improve how the current task and mission objective are shown. Some participant comments suggested that knowing what the agent is trying to do right now may be just as important as knowing where execution is in the Behaviour Tree. This points toward better support for showing mission intent and current planning objectives during runtime.

Finally, future work could explore how the same general visualisation idea could be adapted to systems that do not use Behaviour Trees. The current thesis focuses on a Behaviour Tree based autonomy stack, but some participants also worked with other approaches where the decision process is less directly visible. Those systems are harder to inspect internally, which may make visualisation both more difficult and more necessary.

7 Conclusion

This thesis investigated how visualisation can help when analysing internal autonomy processes in autonomous aircraft simulation. The work focused on which internal representations seem important to show, how they can be visualised, and whether this can help engineers understand system behaviour better in simulation. To do this, the thesis first used pre-study interviews to identify analysis needs. A prototype was then developed in a simulated environment, and after that the prototype was evaluated through a qualitative interview session based on a few simulation runs.

7.1 Research Question 1

The first research question asked which internal representations within autonomous systems are most critical for diagnosing system behaviour in simulation scenarios. Based on the pre-study and evaluation, the most important representations were the current decision state, the execution path through the Behaviour Tree, and the relation between the planned and executed aircraft motion. These were the most central because they helped participants connect what the aircraft did in the environment to what the autonomy system was doing internally.

The Behaviour Tree execution state was especially important. Active nodes, state transitions, node outcomes, and the path through the tree gave participants a way to understand where the system was in its decision process. Planning information and estimation related values were also important, but mainly because they helped explain why the Behaviour Tree entered or did not enter a certain state. In this study, they were therefore secondary to the decision state, but still necessary for interpreting the cause of anomalous behaviour.

The study therefore suggests a priority order. First, engineers need to see the current decision state and execution flow. Second, they need to relate that state to spatial mission information such as planned path, flown path, aircraft position, and no-fly zones. Third, they need selected internal values from planning, estimation, or control that explain why the current decision state was reached. Showing all internal variables was not seen as equally useful, since participants also described the BT HUD as information-heavy.

7.2 Research Question 2

The second research question asked how these internal representations can be structured and visualised to support interpretation of autonomous decision processes in simulation. The study suggests that the information should be structured around the analysis workflow: first showing what happened externally, then showing where the autonomy system was in its decision process, and finally showing selected internal values that may explain why that state was reached.

In the prototype, this was done by combining three linked views. The spatial view showed aircraft motion, planned path, flown path, and no-fly zones. The Behaviour Tree viewer showed active execution state and movement through the tree. The BT HUD showed selected internal values related to planning, estimation, and control. This structure allowed participants to move from external behaviour to internal decision state and then to more detailed internal information.

The results also suggest that internal information should be structured selectively rather than shown as a large set of variables. The BT HUD should therefore be organised around the current analysis task, for example by prioritising current Behaviour Tree state, relevant conditions, active goal or waypoint, planning status, and estimation information related to the current behaviour. Less relevant variables should be hidden, grouped, or made configurable. This would make the visualisation easier to inspect while still keeping important internal information available.

Finally, the information should be structured temporally. Participants wanted replay and stepwise inspection so that Behaviour Tree execution, internal values, and aircraft motion could be followed together over time. This means that the views should not only be placed next to each other, but also be synchronised around the same time.

7.3 Research Question 3

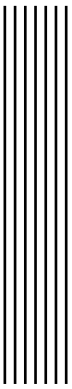
The third research question asked how a structured visualisation approach supports experts in interpreting and reasoning about anomalies in autonomous system behaviour during simulation. This question is the hardest one to answer strongly. The results do suggest that the visualisation helped the participants reason better about the anomalous scenario, but the study was too limited to claim a clear improvement in a stronger sense.

In the anomalous scenario without internal visualisation, participants could make reasonable guesses based on the aircraft motion, especially around coordinates, direction, and internal state. However, those guesses were hard to validate. When the full visualisation was shown, participants could compare their guesses with Behaviour Tree execution and internal values. The visualisation therefore supported validation of reasoning rather than simply giving the answer directly.

The answer to RQ3 should still be seen as exploratory. The evaluation involved a small participant group, a limited set of scenarios, and the same anomalous case was shown twice. Because of that, the findings cannot show a measured improvement in debugging performance. They do however indicate that linked internal and spatial views can make reasoning about anomalous behaviour more grounded and less dependent on unsupported guesses.

7.4 Overall

For engineers and developers working with autonomous systems, the thesis points to the value of making internal state more visible during analysis. The work does not provide a complete general solution, but it does suggest that combining internal execution views with spatial mission context can be useful. The thesis is also limited in several ways. It was qualitative, the participant group was small, the evaluation used only a small number of scenarios, and the prototype focused on a Behaviour Tree-based implementation. Because of that, the results should be seen as exploratory and tied to this specific context.



Bibliography

[1] R. Perez-Segui, P. Arias-Perez, J. Melero-Deza, M. Fernandez-Cortizas, D. Perez-Saura and P. Campoy. 'Bridging the Gap between Simulation and Real Autonomous UAV Flights in Industrial Applications'. In: *Aerospace* 10.9 (2023), p. 814. DOI: 10.3390/aerospace10090814.

[2] Luis D. Barrio, Wojciech Korek, Murat Millidere and James F. Whidborne. 'Analysis of Visualization Systems in Flight Simulators'. In: *Proceedings of the AIAA AVIATION 2023 Forum*. American Institute of Aeronautics and Astronautics, 2023. DOI: 10.2514/6.2023-3476. URL: https://dspace.lib.cranfield.ac.uk/handle/1826/20098.

[3] Sebastian Thrun, Wolfram Burgard and Dieter Fox. *Probabilistic Robotics*. The MIT Press, 2005. ISBN: 9780262201629.

[4] Steve Macenski, Tully Foote, Brian Gerkey, Chris Lalancette and William Woodall. 'Robot Operating System 2: Design, architecture, and uses in the wild'. In: *Science Robotics* 7.66 (2022). DOI: 10.1126/scirobotics.abm6074.

[5] David A. Sparrow, David M. Tate, John C. Biddle, Nicholas J. Kaminski and Poornima Madhavan. *Assessing the Quality of Decision-Making by Autonomous Systems*. Tech. rep. IDA Document P-9116. Institute for Defense Analyses, 2018. URL: https://www.ida.org/-/media/feature/publications/a/as/assessing-the-quality-of-decision-making-by-autonomous-systems/p-9116.ashx.

[6] Pericle Salvini, Tyler Reinmund, Benjamin Hardin, Keri Grieman, Carolyn Ten Holter, Aaron Johnson, Lars Kunze, Alan Winfield and Marina Jirotka. 'Human involvement in autonomous decision-making systems: Lessons learned from three case studies in aviation, social care and road vehicles'. In: *Frontiers in Political Science* 5 (2023). DOI: 10.3389/fpos.2023.1238461.

[7] Daniel A. Keim, Gennady Andrienko, Jean-Daniel Fekete, Carsten Görg, Jörn Kohlhammer and Guy Melançon. 'Visual Analytics: Definition, Process, and Challenges'. In: *Information Visualization: Human-Centered Issues and Perspectives*. Vol. 4950. Lecture Notes in Computer Science. Springer, 2008, pp. 154–175. DOI: 10.1007/978-3-540-70956-5_7.

[8] Bryce Ikeda and Daniel Szafir. 'Advancing the Design of Visual Debugging Tools for Roboticists'. In: *Proceedings of the 2022 ACM/IEEE International Conference on Human-Robot Interaction*. HRI '22. Sapporo, Hokkaido, Japan: IEEE Press, 2022, pp. 195–204. DOI: 10.5555/3523760.3523789.

[9] Rudolf E. Kalman. 'A New Approach to Linear Filtering and Prediction Problems'. In: *Transactions of the ASME–Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.

[10] Nathan Koenig and Andrew Howard. 'Design and use paradigms for Gazebo, an open-source multi-robot simulator'. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2004, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727.

[11] Vadim Engelson. 'Simulation and Visualization of Autonomous Helicopter and Service Robots'. In: *Linköpings Electronic Articles in Computer and Information Science* 5.8 (2000). ISSN: 1401-9841. URL: https://www.ep.liu.se/ea/cis/2000/008/.

[12] Jessica Hullman. 'Why Authors Don't Visualize Uncertainty'. In: *IEEE Transactions on Visualization and Computer Graphics* 26.1 (2020), pp. 130–139. DOI: 10.1109/TVCG.2019.2934647.

[13] Matthew Kay, Tara Kola, Jessica Hullman and Sean A. Munson. 'When (Ish) is My Bus? User-Centered Visualizations of Uncertainty in Everyday, Mobile Predictive Systems'. In: *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems (CHI)*. 2016, pp. 5092–5103. DOI: 10.1145/2858036.2858558.

[14] Suyun Bae, Federico Rossi, Joshua Vander Hook, Scott Davidoff and Kwan-Liu Ma. 'A Visual Analytics Approach to Debugging Cooperative, Autonomous Multi-Robot Systems' Worldviews'. In: *Proceedings of the 2020 IEEE Conference on Visual Analytics Science and Technology (VAST)*. 2020, pp. 24–35. DOI: 10.1109/VAST50239.2020.00008.

[15] Waleed Saeed and Christian W. Omlin. 'Explainable Artificial Intelligence: A Survey of Models, Applications, and Challenges'. In: *Artificial Intelligence Review* 56.4 (2023), pp. 3341–3380. DOI: 10.1007/s10462-022-10285-6.

[16] Fred Hohman, Minsuk Kahng, Robert Pienta and Duen Horng Chau. 'Visual Analytics in Deep Learning: An Interrogative Survey for the Next Frontiers'. In: *IEEE Computer Graphics and Applications* 38.3 (2018), pp. 76–90. DOI: 10.1109/MCG.2018.032421651.

[17] Finale Doshi-Velez and Been Kim. *Towards A Rigorous Science of Interpretable Machine Learning*. 2017. arXiv: 1702.08608 [stat.ML]. URL: https://arxiv.org/abs/1702.08608.

[18] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018. ISBN: 9781138032055.

[19] Alejandro Marzinotto, Michele Colledanchise, Christian Smith and Petter Ögren. 'Towards a Unified Behavior Trees Framework for Robot Control'. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2014, pp. 5420–5427. DOI: 10.1109/ICRA.2014.6907656.

[20] Matteo Iovino, Edvards Scukins, Johan Styrud, Petter Ögren and Christian Smith. 'A Survey of Behavior Trees in Robotics and AI'. In: *Robotics and Autonomous Systems* 154 (2022), p. 104096. DOI: 10.1016/j.robot.2022.104096.

[21] Michele Colledanchise and Petter Ögren. 'How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions'. In: *IEEE Transactions on Robotics* 33.2 (2017), pp. 372–389. DOI: 10.1109/TRO.2016.2633567.

[22] Petter Ögren. 'Increasing Modularity of UAV Control Systems using Computer Game Behavior Trees'. In: *AIAA Guidance, Navigation, and Control Conference*. 2012. DOI: 10.2514/6.2012-4458.

[23] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler and Andrew Y. Ng. 'ROS: an open-source Robot Operating System'. In: *ICRA Workshop on Open Source Robotics*. 2009.

[24] Christophe Bédard, Ingo Lütkebohle and Michel Dagenais. 'ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2'. In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 6511–6518. DOI: 10.1109/LRA.2022.3174346.

[25] Christophe Bédard, Pierre-Yves Lajoie, Giovanni Beltrame and Michel Dagenais. 'Message flow analysis with complex causal links for distributed ROS 2 systems'. In: *Robotics and Autonomous Systems* 161 (2023), p. 104361. DOI: 10.1016/j.robot.2022.104361.

[26] Lorenz Meier, Dominik Honegger and Marc Pollefeys. 'PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms'. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 6235–6240. DOI: 10.1109/ICRA.2015.7140074.

[27] Robert H. Wortham, Andreas Theodorou and Joanna J. Bryson. 'Robot Transparency: Improving Understanding of Robot Behaviour'. In: *Proceedings of the ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. ACM, 2017, pp. 430–438. DOI: 10.1145/2909824.3020247.

[28] Bradley Hayes and Julie A. Shah. 'Improving Robot Controller Transparency Through Autonomous Policy Explanation'. In: *Proceedings of the ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. ACM, 2017, pp. 303–312. DOI: 10.1145/2909824.3020233.

[29] Emil Stubbe Kolvig-Raun, Miguel Campusano and Thor Malmby Jorgin. 'Contextual visualizations for debugging collaborative robots'. In: *Proceedings of the 2024 ACM/IEEE 6th International Workshop on Robotics Software Engineering (RoSE '24)*. 2024, pp. 31–34. DOI: 10.1145/3643663.3643965.

[30] Emil Stubbe Kolvig-Raun, Mikkel Baun Kjærgaard and Ralph Brorsen. 'EDDE: An Event-Driven Data Exchange to Accurately Introspect Cobot Applications'. In: *Proceedings of the 2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE 2023)*. 2023, pp. 25–30. DOI: 10.1109/RoSE59155.2023.00009.

[31] Zihang Li, Atsushi Hasegawa and Takuya Azumi. 'Autoware_Perf: A tracing and performance analysis framework for ROS 2 applications'. In: *Journal of Systems Architecture* 123 (2022), p. 102341. DOI: 10.1016/j.sysarc.2021.102341.

[32] Takahisa Kuboichi, Atsushi Hasegawa, Bo Peng, Keita Miura, Kenji Funaoka, Shinpei Kato and Takuya Azumi. 'CARET: Chain-Aware ROS 2 Evaluation Tool'. In: *2022 IEEE International Conference on Embedded and Ubiquitous Computing (EUC)*. 2022, pp. 1–8. DOI: 10.1109/EUC57774.2022.00010.

[33] Razan Ghzouli, Thorsten Berger, Einar Broch Johnsen, Swaib Dragule and Andrzej Wąsowski. 'Behavior Trees in Action: A Study of Robotics Applications'. In: *Proceedings of the 13th ACM SIGPLAN International Conference on Software Language Engineering*. ACM, 2020, pp. 196–209. DOI: 10.1145/3426425.3426942.

[34] Michele Colledanchise, Giuseppe Cicala, Daniele E. Domenichelli, Lorenzo Natale and Armando Tacchella. 'Formalizing the Execution Context of Behavior Trees for Runtime Verification of Deliberative Policies'. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021, pp. 9841–9848. DOI: 10.1109/IROS51168.2021.9636129.

[35] Shreyansh Daftry, Sam Zeng, J. Andrew Bagnell and Martial Hebert. 'Introspective Perception: Learning to Predict Failures in Vision Systems'. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 1743–1750. DOI: 10.1109/IROS.2016.7759279.

[36] Virginia Braun and Victoria Clarke. 'Using thematic analysis in psychology'. In: *Qualitative Research in Psychology* 3.2 (2006), pp. 77–101. DOI: 10.1191/1478088706qp063oa.

A Interview Questions

A.1 Pre-Study Interview Questions

- Vad jobbar du med idag?
- Hur länge har du arbetat med det?
- Hur tolkar du begreppet autonomi i det du jobbar med?
- När ett autonomt beteende ser fel ut i simulering, hur går du tillväga för att analysera det?
- Vilka interna tillstånd eller signaler tittar du oftast på vid felsökning?
- Finns det interna tillstånd som är viktiga men svåra att tolka i dagens visualiseringar?
- Hur viktigt är det för dig att se osäkerhet eller confidence i visualiseringar? Hur skulle du vilja att den informationen presenteras?
- Om du fick välja tre saker kring visualisering av autonoma flygplan som borde visualiseras bättre, vilka skulle det vara?
- Om vi tänker framåt i utvecklingen för autonoma flygplan, finns det någon form av visualisering som kan vara intressant framöver?

A.2 Evaluation Session Questions

A.2.1 Scenario 1: Nominal Behaviour with Full Visualization

- Vilken information i detta scenario är viktigast för att förstå vad systemet gör?
- Vilka delar hjälper eller fungerar som bra visuella hjälpmedel?
- Är det något i visualiseringen som är oklart?
- Hur upplevs BT-viewern?
- Hur upplevs BT-HUD:en?

- Är det för mycket information som visas samtidigt?
- Vad är det som händer och vad är det som ska hända?
- Finns det något som skulle kunna läggas till för att underlätta analysen?

A.2.2 Scenario 2: Anomalous Behaviour without Additional Visualization

- Baserat på vad ni ser, vad tror ni är fel? Det kan vara fler än en sak.
- Vad hade ni velat kunna se för att förstå problemet bättre?
- Vad hade ni tittat på härnäst om detta varit en verklig analys?

A.2.3 Scenario 3: Anomalous Behaviour with Full Visualization

- Baserat på vad ni ser nu, vad tror ni är fel?
- Vad blev tydligare när den extra visualiseringen visades?
- Vilka visuella delar hjälpte mest för att förstå eller analysera problemet?
- Finns det något som fortfarande är oklart?
- Gick det att validera eller förkasta tidigare gissningar med hjälp av visualiseringen?
- Vilka delar saknas fortfarande för att kunna förstå beteendet bättre?

A.2.4 General Follow-Up Questions

- Vilka delar av det ni såg var viktigast att se?
- Vad var minst viktigt att se?
- Hjälpte det visuella att förstå vad som hände, varför det hände, eller båda?
- Vad hade ni helst lagt till för att underlätta i ert eget arbete?
- Om denna produkt hade fortsatt utvecklas, vilka förbättringar hade ni helst sett?
- Skulle samma grundidé kunna vara användbar även för andra typer av autonomi-arkitekturer?